

## Step 1 — Install Required Libraries

```

!pip install chromadb
!pip install sentence-transformers
!pip install langchain

39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (7.2 kB)
:chromadb)
:one-any.whl.metadata (5.8 kB)
pydantic>=2.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (2.12.3)
pydantic-settings>=2.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (2.13.1)
m chromadb)
312-cp312-manylinux1_x86_64.manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_5_x86_64.whl.metadata (8.7 kB)
uvicorn>=0.18.3 in /usr/local/lib/python3.12/dist-packages (from uvicorn[standard]>=0.18.3->chromadb) (0.42.0)
numpy>=1.22.5 in /usr/local/lib/python3.12/dist-packages (from chromadb) (2.0.2)
typing-extensions>=4.5.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (4.15.0)
(from chromadb)
.4-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (5.2 kB)
opentelemetry-api>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (1.38.0)
:er-otlp-proto-grpc>=1.2.0 (from chromadb)
:orter-otlp-proto-grpc-1.40.0-py3-none-any.whl.metadata (2.6 kB)
opentelemetry-sdk>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (1.38.0)
tokenizers>=0.13.2 in /usr/local/lib/python3.12/dist-packages (from chromadb) (0.22.2)
n chromadb)
2.py3-none-any.whl.metadata (51 kB)
52.0/52.0 kB 1.8 MB/s eta 0:00:00
tqdm>=4.65.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (4.67.3)
overrides>=7.3.1 in /usr/local/lib/python3.12/dist-packages (from chromadb) (7.7.0)
importlib-resources in /usr/local/lib/python3.12/dist-packages (from chromadb) (6.5.2)
grpcio>=1.58.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (1.78.0)
chromadb)
3-abi3-manylinux_2_34_x86_64.whl.metadata (10 kB)
:typer>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (0.24.1)
(from chromadb)
3-py2.py3-none-any.whl.metadata (1.7 kB)
tenacity>=8.2.3 in /usr/local/lib/python3.12/dist-packages (from chromadb) (9.1.4)
pyyaml>=6.0.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (6.0.3)
mmh3>=4.0.1 in /usr/local/lib/python3.12/dist-packages (from chromadb) (5.2.1)
orjson>=3.9.12 in /usr/local/lib/python3.12/dist-packages (from chromadb) (3.11.7)
httpx>=0.27.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (0.28.1)
rich>=10.11.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (13.9.4)
jsonschema>=4.19.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (4.26.0)
packaging>=24.0 in /usr/local/lib/python3.12/dist-packages (from build>=1.0.3->chromadb) (26.0)
m build>=1.0.3->chromadb)
L.2.0-py3-none-any.whl.metadata (1.3 kB)
anyio in /usr/local/lib/python3.12/dist-packages (from httpx>=0.27.0->chromadb) (4.12.1)
certifi in /usr/local/lib/python3.12/dist-packages (from httpx>=0.27.0->chromadb) (2026.2.25)
httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx>=0.27.0->chromadb) (1.0.9)
idna in /usr/local/lib/python3.12/dist-packages (from httpx>=0.27.0->chromadb) (3.11)
h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx>=0.27.0->chromadb) (0.16.0)
attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=4.19.0->chromadb) (25.4.0)
jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=4.19.0->chromadb) (2025.9.1)
referencing>=0.28.4 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=4.19.0->chromadb) (0.37.0)
rpds-py>=0.25.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=4.19.0->chromadb) (0.30.0)
six>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->chromadb) (1.17.0)
python-dateutil>=2.5.3 in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->chromadb) (2.9.0.post0)
websocket-client!=0.40.0,!0.41.*,!0.42.*,>=0.32.0 in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->chromadb) (1.7.0)
requests in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->chromadb) (2.32.4)
requests-oauthlib in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->chromadb) (2.0.0)
urllib3!=2.6.0,>=1.24.2 in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->chromadb) (2.5.0)
m kubernetes>=28.1.0->chromadb)
y3-none-any.whl.metadata (340 bytes)

```

## Step 2 — Import Required Libraries

```

import chromadb
from sentence_transformers import SentenceTransformer

```

## Step 3 — Initialize ChromaDB Client

```

client = chromadb.Client()

collection = client.create_collection(name="rag_collection")

```

## Step 4 — Load the Embedding Model

```
model = SentenceTransformer('all-MiniLM-L6-v2')
```

/usr/local/lib/python3.12/dist-packages/huggingface\_hub/utils/\_auth.py:94: UserWarning:  
The secret `HF\_TOKEN` does not exist in your Colab secrets.  
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it  
You will be able to reuse this secret in all of your notebooks.  
Please note that authentication is recommended but still optional to access public models or datasets.

```
warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
modules.json: 100% 349/349 [00:00<00:00, 27.4kB/s]

config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 9.70kB/s]

README.md: 10.5k/? [00:00<00:00, 751kB/s]

sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 4.22kB/s]

config.json: 100% 612/612 [00:00<00:00, 53.7kB/s]

model.safetensors: 100% 90.9M/90.9M [00:01<00:00, 454MB/s]

Loading weights: 100% 103/103 [00:00<00:00, 411.45it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key | Status | |
-----+-----+--
embeddings.position_ids | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tokenizer_config.json: 100% 350/350 [00:00<00:00, 14.5kB/s]

vocab.txt: 232k/? [00:00<00:00, 6.81MB/s]

tokenizer.json: 466k/? [00:00<00:00, 21.1MB/s]

special_tokens_map.json: 100% 112/112 [00:00<00:00, 5.80kB/s]

config.json: 100% 190/190 [00:00<00:00, 18.2kB/s]
```

## Step 5 — Prepare Sample Documents

```
documents = [
    "Artificial Intelligence is the simulation of human intelligence.",
    "Machine Learning is a subset of AI.",
    "Deep Learning uses neural networks.",
    "Natural Language Processing deals with text data.",
    "RAG combines retrieval and generation.",
    "Neural networks are inspired by the human brain.",
    "AI is used in healthcare, finance and robotics.",
    "Supervised learning uses labeled data.",
    "Unsupervised learning finds hidden patterns.",
    "Reinforcement learning uses reward based learning."
]

ids = ["doc1", "doc2", "doc3", "doc4", "doc5", "doc6", "doc7", "doc8", "doc9", "doc10"]
```

## Step 6 — Convert Text to Embeddings

```
embeddings = model.encode(documents)
```

## Step 7 — Store Embeddings in ChromaDB

```
collection.add(
    documents=documents,
    embeddings=embeddings,
    ids=ids
)
```

## Step 8 — Perform Similarity Search

```

query = "What is Artificial Intelligence?"

query_embedding = model.encode([query])

results = collection.query(
    query_embeddings=query_embedding,
    n_results=2
)

print(results)

```

```
{'ids': [['doc1', 'doc7']], 'embeddings': None, 'documents': [['Artificial Intelligence is the simulation of human intelligence.
```

### Step 9 — Display Retrieved Documents

```

for doc in results['documents'][0]:
    print(doc)

```

```

Artificial Intelligence is the simulation of human intelligence.
AI is used in healthcare, finance and robotics.

```

### Step 10 — Build Simple Retrieval Function

```

def retrieve(query):
    query_embedding = model.encode([query])

    results = collection.query(
        query_embeddings=query_embedding,
        n_results=2
    )

    return results['documents'][0]

print(retrieve("Explain machine learning"))

```

```
['Machine Learning is a subset of AI.', 'Supervised learning uses labeled data.']
```

### Step 11 — Test with Different Queries

```

print(retrieve("What is deep learning"))
print(retrieve("Explain neural networks"))
print(retrieve("What is NLP"))

```

```

['Deep Learning uses neural networks.', 'Machine Learning is a subset of AI.']
['Neural networks are inspired by the human brain.', 'Deep Learning uses neural networks.']
['Natural Language Processing deals with text data.', 'Supervised learning uses labeled data.']

```